

Sistemas Operativos II

PRÁCTICA I

Mario VILAR
20392724

David DíEZ
20392643

27 de septiembre de 2023



UNIVERSITAT DE
BARCELONA

ÍNDICE

1	Introducción	2	<i>Ejercicio 9</i>	3
2	Pruebas	2	<i>Ejercicio 10</i>	4
	<i>Ejercicio 1</i>	2	<i>Ejercicio 11</i>	4
	<i>Ejercicio 2</i>	2	<i>Ejercicio 12</i>	4
	<i>Ejercicio 3</i>	2	<i>Ejercicio 13</i>	4
	<i>Ejercicio 4</i>	3	<i>Ejercicio 14</i>	4
	<i>Ejercicio 5</i>	3	<i>Ejercicio 15</i>	5
	<i>Ejercicio 6</i>	3	<i>Ejercicio 16</i>	5
	<i>Ejercicio 7</i>	3	3 Conclusiones y contribuciones	5
	<i>Ejercicio 8</i>	3	<i>Figura 3.1</i>	5

1

INTRODUCCIÓN

Una de las amenazas más comunes en la seguridad de los sistemas operativos, incluido Linux, es la vulnerabilidad conocida como *stack overflow*. Esta vulnerabilidad ocurre cuando un programa sobrepasa los límites de la memoria asignada al stack, lo que puede ser aprovechado por atacantes para obtener acceso no autorizado al sistema. En esta práctica vamos a investigar este tipo de vulnerabilidades, por qué se producen y cómo pueden ser realizadas (stack4 y stack5) y evitadas.

2

PRUEBAS

Ejercicio 1. *Mediante líneas de comandos, ¿cómo puedes mostrar los usuarios y grupos del sistema?*

Respuesta. Podemos usar el comando `cat` junto con el archivo `/etc/passwd`, que contiene información sobre los usuarios del sistema, para mostrar una lista de lo que nos interesa encontrar. El comando siguiente: `cat /etc/passwd | grep -i "oslabadmin"` nos devuelve las ocurrencias relacionadas con el `oslabadmin`:
`oslabadmin:x:1000:1000:oslabadmin:/home/oslabadmin:/bin/bash`.

Caben destacar el tercer y el cuarto atributo, que nos dan el UID y GID, respectivamente. Los grupos del sistema funcionan con la estructura similar `cat /etc/group | grep -i "oslabadmin"`. Lo que devuelve este comando es todos los grupos en que `oslabadmin` es un usuario, por ejemplo: `sudo:x:27:oslabadmin`. El grupo en cuestión se llama `sudo`, el número 27 representa el ID de grupo, y `oslabadmin` es el único usuario que pertenece a este grupo, lo que permite ejecutar comandos con privilegios de administrador. ✓

Ejercicio 2. *¿Qué ha pasado? ¿Para qué sirve el comando su? ¿En qué situaciones sería interesante utilizarlo? ¿Qué pasa si abro una nueva terminal?*

Respuesta. Ejecutando `man su` vemos que:

The su utility requests appropriate user credentials via PAM and switches to that user ID (the default user is the superuser). A shell is then executed.

A priori parece que se abrirá una nueva pestaña desde el usuario `oslabadmin` después, claro está, de poner la contraseña del usuario que queremos usar. Sin embargo, es en esa misma sesión de terminal que podríamos ejecutar comandos con permisos de administrador (`oslabadmin@oslab`). `su` sería interesante para ejecutar tareas que un usuario normal no está autorizado a hacer, como acceder a ficheros únicamente accesibles por el administrador. Y en último lugar, si abrimos una nueva terminal perdemos los permisos otorgados por `su`, ya que solo prevalecen en la sesión que inicia el mismo comando. ✓

Ejercicio 3. *¿Cómo podemos ver las propiedades del documento creado (ejemplo.txt)? ¿Quién es el propietario del archivo? ¿A qué grupo pertenece? ¿Qué permisos tienen el propietario, grupo y otros usuarios sobre este archivo?*

Respuesta. Lo podríamos ver, por ejemplo, ejecutando el comando `ls -l` en la misma carpeta `/tmp`, a la cual accedemos desde el directorio raíz. El propietario de este archivo es `oslabadmin`, ya que seguimos en la misma ventana que el ejercicio 2. En nuestro caso, el grupo sería también `oslabadmin`. También tendríamos otra información, como el tamaño y la fecha de modificación. En particular, en primera instancia nos aparecen los permisos. `-rw-r-r-` indica que el archivo es legible y escribible por el propietario, pero solo legible por otros usuarios. ✓

Ejercicio 4. *¿Qué pasa al intentar guardar las modificaciones? ¿Por qué?*

Respuesta. Nos salta una advertencia, requiriendo autenticación. Como Ubuntu interpreta que estamos corriendo Sublime como usuario, lo cual es cierto ya que nos hemos autenticado como usuarios al iniciar sesión, no podemos guardar los cambios en tal archivo protegido sin los permisos de superusuario. ✓

Ejercicio 5. *¿Puedes ver el contenido del fichero? ¿Por qué? ¿Qué acción se ha realizado con el `chmod 700 ./ejemplo.txt`?*

Respuesta. No podemos ver el contenido del fichero, aparece como si estuviera vacío de hecho, ya que `chmod 700 ./ejemplo.txt` ha dejado sin permisos tanto de escritura como de lectura a `oslab`, que es el usuario responsable de abrir Sublime. Mientras tanto, `oslabadmin` sigue gozando de todos los permisos correspondientes. ✓

Ejercicio 6. *¿Qué comanda utilizarías?*

Respuesta. Si utilizáramos `chmod 700 ./S02` como en el ejercicio 5 no estaríamos permitiendo acceder, ni leer, ni escribir al grupo de `oslabadmin`. Por lo tanto, la respuesta correcta es `chmod 770 ./S02` (equivalente a `rw-rwx--`). ✓

Ejercicio 7. *¿Cómo cambiarías estos permisos para permitir la ejecución del ejecutable?*

Respuesta. No hemos tenido problema para ejecutarlo, ya que los permisos son muy permisivos (valga la redundancia). Sin embargo, hemos protegido el script como en el ejercicio 5 y es entonces cuando no podemos ejecutar, con el error `bash: script.sh: Permission denied`. Ahora, de la misma manera que en el ejercicio 6, debemos cambiar los permisos con `chmod 770 script.sh`, previamente cambiando al usuario `oslabadmin` mediante su `oslabadmin`. ✓

Ejercicio 8. *¿A qué es debido este mensaje? ¿Por qué aparece con “SistemasOperativos” y no para las otras contraseñas?*

Respuesta. El mensaje que obtenemos es `*** stack smashing detected ***: terminated Aborted (core dumped)`. El motivo de ello es porque estamos utilizando un `buffer` de 15 elementos y, en particular, la palabra `SistemasOperativos` tiene 20 letras. Por ello, estamos intentando poner más de lo que cabe dentro del `buffer`. Esto no sucede con `Chocolate` ni con `CafeConLeche` ya que tienen menos de 15 letras. ✓

Ejercicio 9. *Para la contraseña de “SistemasOperativos”, ¿Cómo podrías evitar ese `core dumped`? Analiza el comportamiento del programa para las diferentes contraseñas anteriores y justifica dicho comportamiento.*

Demostración. La solución es simple, debemos cambiar la longitud del buffer. Podemos hacerlo hardcoded y poner un 20, podemos utilizar `strlen('SistemasOperativos')` o `strlen()` del input que reciba el programa. Esta última es la más fiable ya que funcionará para cualquier caso. ✓

Ejercicio 10. *Describe que hace el exploit.*

Respuesta. En la primera línea crea la letra «A» en formato hexadecimal. En las siguientes líneas multiplica por 64 y 8 la cantidad de «A» que aparecen en la variable `exploit`. Al final del todo habrán 88 letras «A». Por último añade una secuencia de bytes a dicha variable en ASCII hexadecimal, la dirección de retorno. ✓

Ejercicio 11. *¿Por qué no funciona con python3 y si con python2?*

Respuesta. El problema por el cual no se ejecuta en python3 es que faltan unos paréntesis en el `print`. Para poder ejecutar este código habría que sustituir `print exploit` por `print(exploit)`. ✓

Ejercicio 12. *Dicha dirección de la función, ¿se debe proporcionar en Big Endian o Little Endian? ¿Cuál es la diferencia entre ellas? ¿Como puedes averiguar si la arquitectura de la máquina usa uno u otro?*

Respuesta. La principal diferencia entre *Big Endian* y *Little Endian* es la forma de organizar los bytes en la memoria. Por un lado, *Little Endian* escribe el byte menos significativo a en la dirección de memoria más baja. *Big Endian* lo hace al revés. Hemos hecho un *script* para comprobar que nuestra máquina es, en efecto, *Little Endian*:

```
1  #include <stdio.h>
2  int main() {
3      unsigned int valor = 1; unsigned char *byte = (unsigned char *)&valor;
4      if (*byte == 1) { printf("La máquina es little-endian."); }
5      else { printf("La máquina es big-endian."); }
6      return 0;
7  }
```

La dirección se debe proporcionar, por tanto, en *Little Endian*. ✓

Ejercicio 13. *Describe el proceso realizado para explotar correctamente el fichero y que salga el mensaje de Congratulations, you've finished phoenix/stack-four :) Well done!*

Respuesta. Primeramente utilizamos el comando `readelf -s stack4.exe | grep "complete_level"` para ver en que dirección de memoria está la función `complete_level`, nos fijamos en la columna `Value` (que contiene la dirección en memoria donde se encuentra el símbolo) y nos lo quedamos. Una vez obtenemos este valor lo escribimos en `stack4_exploit.py`. La idea de esto es poder desbordar el stack hasta la posición de memoria asignada a `complete_level`. En nuestro caso, la mencionada dirección es `00000000000011c9`, por lo que, teniendo en cuenta que trabajamos en *Little Endian*, cambiando los bytes de orden y a formato ASCII hexadecimal, queda así: `\xc9\x11\x00\x00\x00\x00\x00\x00`. ✓

Ejercicio 14. *¿Qué puedes decir el rango de direcciones y permisos de la pila?*

Respuesta. Para encontrar el PID del proceso hemos utilizado `ps aux | grep "/stack5.exe"`. Una vez ejecutado el comando para nuestro PID con valor 3334, `cat /proc/3334/maps | grep "stack"`, hemos visto que el rango de direcciones es de `7ffdf28f6000` a `7ffdf2917000` y los permisos son `rw-p`. ✓

Ejercicio 15. Describe el proceso para desactivar estas medidas de seguridad.

Respuesta. Para permitir los permisos de ejecución utilizamos `execstack -s ./stack5.exe`. Luego, para desactivar el ASLR utilizamos la línea `setarch --addr-no-randomize ./stack5.exe`. En los dos comandos se aconseja usar `man` para ver qué opciones se deben usar y cómo formularlos. ✓

Ejercicio 16. ¿Qué código se ha inyectado? ¿Dónde está presente esta inyección en código en `stack5_exploit.py`?

Respuesta. Ejecutando la tubería necesaria, encontramos que nuestra dirección de buffer es `0x7fffffffdf0`, es decir, `\xf0\xdf\xff\xff\xff\x7f`. En efecto, modificando la dirección de retorno en el exploit nos encontramos con el resultado del código inyectado. Para saber qué código concretamente se ha inyectado copiamos la siguiente línea de `stack5_exploit.py`:

```
\x48\x31\xd2\x48\xbb\xff\x2f\x62"\x69\x6e\x2f\x6c\x73\x48\xc1\xeb\x08\x53
\x48\x89\xe7\x48\x31\xc0\x50\x57\x48\x89\xe6\xb0\x3b\x0f\x05
```

y la ponemos en el enlace de Hex Works. El resultado que obtenemos es que el código inyectado es un `ls` en la carpeta `bin`. ✓

3

CONCLUSIONES Y CONTRIBUCIONES

En esta práctica hemos aprendido que la prevención de ataques de desbordamiento de pila es esencial, y se puede hacer para llegar a diferentes resultados, como lo son ejecutar una parte no deseada del código o bien inyectar un código propio (malicioso o no). La implementación de medidas de seguridad son pasos críticos para mitigar los riesgos asociados con estas vulnerabilidades.

Los dos hemos hecho más o menos lo mismo, ya que quedamos para resolver la práctica conjuntamente. Cada uno está al tanto de todos los pasos de gestación de la misma.



Figura 3.1: Invitación que le hice llegar a David para poder quedar en su propia casa.